株式会社 ABC　様

「EC サイト」システムの
テストレポート

20XX.4.27

1. 件名

EC サイトのシステムテスト作業

2. 作業内容

(1)作業概要

テスト仕様書に基づいて機能テスト及びシステムテスト作業を行った。

(2)作業項目

・発見されたバグを BTS に追記する。

・毎日、バグの状態分布を集計して分析を行う。

(3)成果物

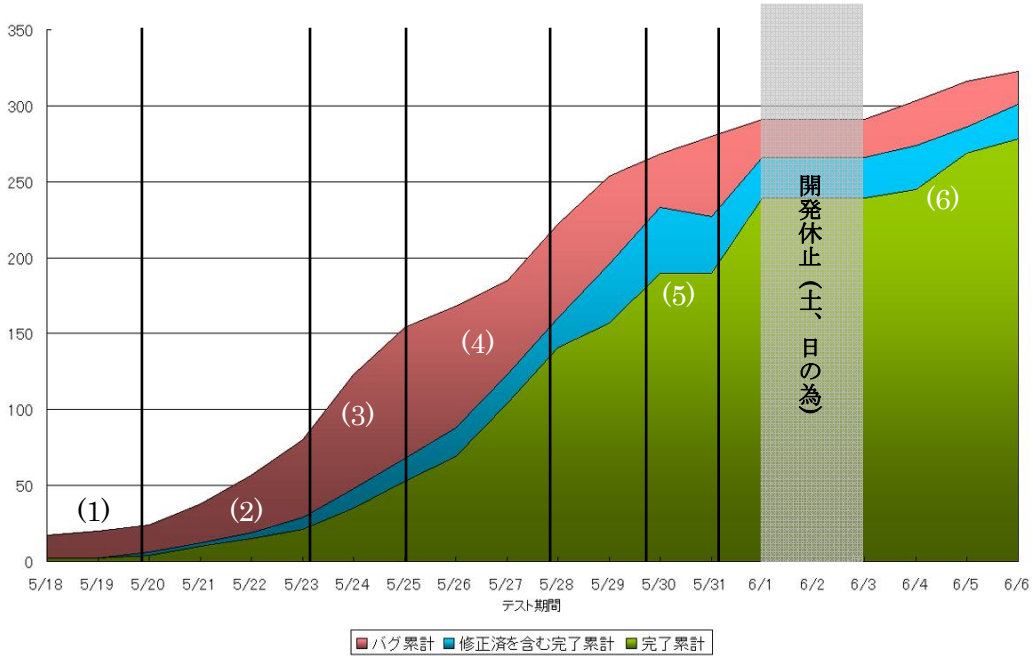
テスト仕様書(PC 版,携帯版)

3. テスト環境

(1)OS	:WindowsXP Professional SP2
(2)ブラウザ	:Internet Explorer Ver.6 SP2/ Firefox2.0.0.3
(3)PC	:DELL OPTIPLEX745
(4)携帯端末	:Softbank / au / docomo

4. 統括

4．1　EC サイト開発におけるバグの顕在化と改修状況



【ポイント】

（1） 5/18-20 ： バグ検出数が少ない

→ テスト準備期間の必要性

本来テスト初期段階においては、バグを数多く検出するのがセオリーだが、本テストにおいては、テスト開始から数日に渡り、バグ検出数が少なかった。

その原因は、テストメンバー（3人）は、システムのトレーニングや仕様の把握を、この時期に、テストを実施しながら行なっており、順調なテスト実施が行えていなかったためである。

仕様・操作方法の把握などは想像以上に時間を要し、テスト効率やテスト内容に大きく影響するものである。テスト実施の前に「テスト準備」の期間を設け、テスト実施とは別に時間を確保することにより、テスト実施期間においては、テスト項目の消化とバグの検出に集中できるテスト体制にしていく。

(2) 5/21-23 ： 修正が追いついていない

→ バグ検出から完了までのタイムラグを少なくする

当該プロジェクトでは開発と修正を同時に行っている為、この期間においては開発を優先としている為、修正が追いついていない。

修正されないバグが残ったまま、テスト実施を進めていくことは、望ましくないことである。それらのバグが与える品質的影響度や、修正後に再度テストを行なわければならないテスト効率性の観点から、バグは検出後、できるだけ早く処置を施すことが望ましい。

テスト実施段階の進捗管理において、上のグラフの赤色の面積ができるだけ小さくなるよう、また、バグ累計のグラフ線から一定の幅で推移するよう、日々確認し、修正作業の遅れを把握することが求められる。

(3) 5/24-5/25 ： バグ検出数の大幅増加

影舞で管理する以前に Excel で管理していた不具合報告を移行したため、管理上のバグ累計がこの期間に大幅に増加している。実質的なバグの検出は (2)の期間に発生しており、(1)で問題にしたバグ検出数の立ち上がりは、実際にはもう少し早かったと推定される。

しかし、逆に言えば、余計に(2)の問題点が強調されることになる。

(4) 5/26-5/28 ： バグ検出数の減少

→ テスト項目あたりのバグ検出率で品質状況を判断

この期間は修正が重点的に実施された為、テストにおいても修正確認テストが主だった（よって完了数が増加している）為、バグ検出数が減少した。

バグ検出数の減少と完了数の増加だけでは品質状況の判断はできず、潜在リスクは解消していない。（その証拠に 5/28 には再度バグ検出数が増加している。）

（修正確認テストではない）「新たに実施したテスト項目あたりのバグ検出率」が減少しているようであれば、品質は改善されているといえる。

また、「修正確認テスト項目あたりのバグ検出率」により、修正作業の品質も測ることができる。

(5) 5/30,5/31 ： 完了数の停滞

6/1 のレビューに向け、テストチームとしては 5/30 にバグ発見に重点を置き、5/31 に修正確認に重点をおいた。その為、上記のグラフ推移状況となった。

(6) 6/3- ： バグが収束する傾向が見えない

→ バグが収束する開発プロセスではない

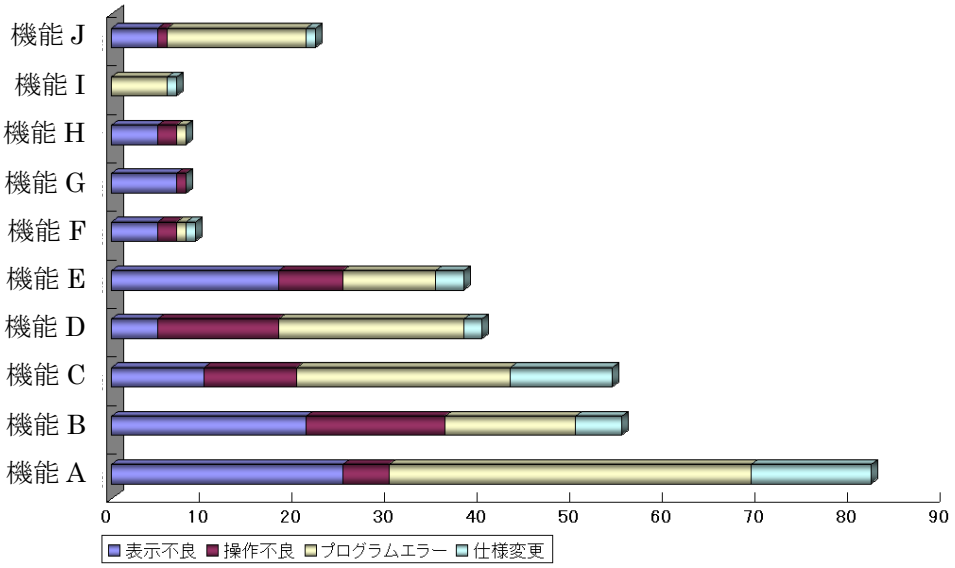
6/1 のレビューの結果、幾つかの仕様変更があり、それに伴ってバグ検出数が増加傾向を示している。また、6/13 納期予定の注文変更・キャンセル機能、定期購入機能の 2 機能についてのテストを始めたため、バグ検出数が上った。

仕様変更や新たなテストを実施すればバグ検出数は増加することは当然のことであり、そういう意味では、テスト作業自体は適切に行われていると言える。

一般的には、バグが収束することが、テスト終了の条件となるが、当該プロジェクトにおいては、仕様変更や新たなテストを随時行うことは、プロジェクトとして認められているため、収束しないことは特に問題とはならない。（もし、品質への影響度を科学的にテストせず、

安易に仕様変更が加えられているならば、そのようなプロジェクト運営自体を変える必要があり、それはテストレポートの範疇ではないと考える）
こういった場合には、バグ修正によってデグレードを起こさないこと、バグ修正による影響範囲を正しく見極め、修正確認テストの確実に行うことが、品質を向上に大きく寄与する。
なお、当該プロジェクトにおいて 5 月末から CVS を導入し、ファイル管理を行っているがそれにより一部のファイルにおいてデグレードが見られたことを付記しておく。

機能（エリア）別バグの割合状況



表示不良　：　画像表示エラー、文言エラーなど画面表示に関する不良。

操作不良　：　画面遷移エラー、戻り操作、など操作に関する不良。

プログラムエラー　：　プログラムが引き起こしたと考えられる不良。

仕様変更　：　仕様変更があったもの、または仕様として処理したもの。

【ポイント】
バグの多い機能と、少ない機能に大別でき、表示不良とプログラムエラーが目立つ。

（1）バグの多い機能

・機能 A

メインとなる機能であり、複雑な要件も多いため、一番多くなっている。

・機能 B

・機能 C

6/1 レビュー後の仕様変更に伴ってバグ発見数が増加した。

・機能 D

・機能 E

・機能 J

(2)バグの少ない機能

・機能 F

・機能 G

・機能 H

・機能 I

(3)バグの分類

「バグの多い機能」については、どの分類のバグも一定の割合で検出されている。

特に下記のバグの分類について注記する。

・表示不良

機能的に致命的なエラーは少ないと思われるが、最も目立つ部分であり、単なる誤記にとどまらず、サービス全体の信用度を低下させる要因になる。

・プログラムエラー

中でもポイント計算、プレゼント付加のプログラムエラーが多い。

一般的にはプログラムエラーは、当該テスト以前に対処しておくべきものであり、プログラムエラー検出のためのテストを、当該テストとは別に実施すべきである。

開発とテストチーム連係によって、品質を向上させることは言うまでもないが、それはきちんとした計画がなされて実現するものである。

各テストプロセスにおいて、どういったバグを検出するのか、段階的な品質時向上のためのテストプロセスの構築やテスト戦略をあらかじめ行う必要がある。

とくに「その他機能」と「検索機能」でプログラムエラーの割合が高く、原因を分析し対処することで、それらの機能の品質を一気に向上させることができると思われる。